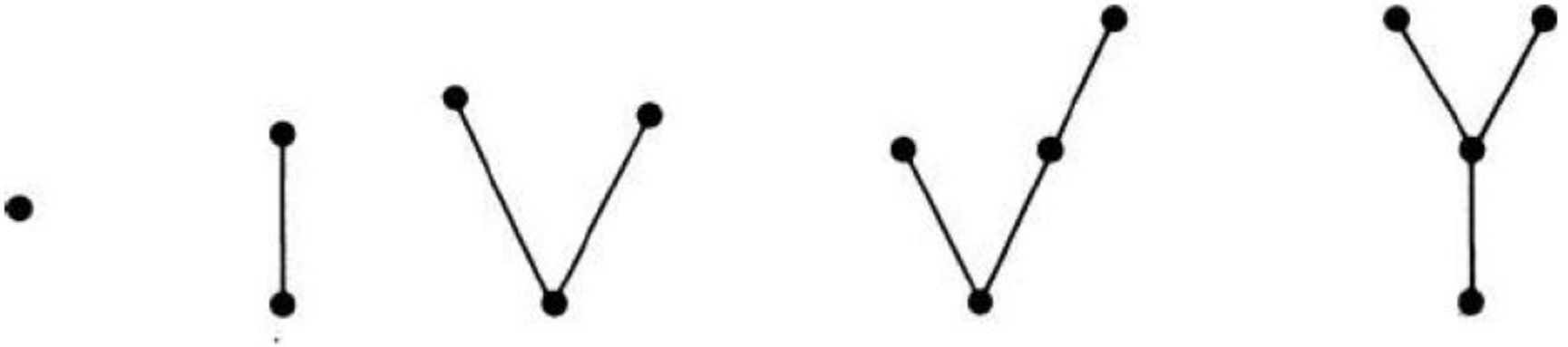


Trees and Fundamental Circuits

TREES

- A connected graph without any circuits
- It follows immediately from the definition that a tree has to be a simple graph, that is, having neither a self-loop nor parallel edges (because they both form circuits).



THEOREM 3-1

“There is one and only one path between every pair of vertices in a tree, T ”

Since T is a connected graph, there must exist at least one path between every pair of vertices in T .

Now suppose that between two vertices a and b of T there are two distinct paths. The union of these two paths will contain a circuit and T cannot be a tree.

Hence the theorem.

THEOREM 3-2

“If in a graph G there is one and only one path between every pair of vertices, G is a tree”.

Existence of a path between every pair of vertices assures that G is connected.

A circuit in a graph (with two or more vertices) implies that there is at least one pair of vertices a, b such that there are two distinct paths between a and b .

Since G has one and only one path between every pair of vertices, G can have no circuit. Therefore, G is a tree.

THEOREM 3-3

“A tree with n vertices has $n - 1$ edges”

The theorem will be proved by induction on the number of vertices.

We can see that the theorem is trivially true for $n = 1, 2$, and 3 . Assume that the theorem holds for all trees with **fewer than n** vertices.

We need to prove that the theorem is also true for n vertices.

THEOREM 3-3

Let us now consider a tree T with n vertices.

In T let e_k be an edge with end vertices v_i and v_j . According to **Theorem 3-1**, there is no other path between v_i and v_j except e_k . Therefore, deletion of e_k from T will disconnect the graph.

Furthermore, $T - e_k$ consists of exactly two components, and since there were no circuits in T to begin with, each of these components is a tree.

THEOREM 3-3

Both these trees, t_1 and t_2 , have fewer than n vertices each, and therefore, by the induction hypothesis, each contains one less edge than the number of vertices in it.

If t_1 contains n_1 vertices and t_2 contains n_2 vertices, then

$$n_1 + n_2 = n$$

And number of edges in t_1 and t_2 is respectively $n_1 - 1$ and $n_2 - 1$, so total edges in $T - e_k$ is

$$n_1 - 1 + n_2 - 1 = n - 2$$

Therefore, T must have had exactly $n - 1$ edges, hence the theorem.

THEOREM 3-5

“A graph is a tree if and only if it is minimally connected”

A connected graph is said to be minimally connected if removal of any one edge from it disconnects the graph.

A minimally connected graph cannot have a circuit; otherwise, we could remove one of the edges in the circuit and still leave the graph connected. Thus a minimally connected graph is a tree.

Conversely, if a connected graph G is not minimally connected, there must exist an edge e_i in G such that $G - e_i$ is connected. Therefore, e_i is in some circuit, which implies that G is not a tree.

THEOREM 3-6

“A graph G with n vertices, $n - 1$ edges, and no circuits is connected”

Suppose there exists a circuitless graph G with n vertices and $n - 1$ edges which is disconnected. In that case G will consist of two or more circuitless components.

Let G consist of two components, g_1 and g_2 . Add an edge e between a vertex v_1 in g_1 and v_2 in g_2 .

Since there was no path between v_1 and v_2 in G , adding e did not create a circuit. Thus $G \cup e$ is a circuitless, connected graph (i.e., a tree) of **n vertices and n edges**, which is not possible, because of **Theorem 3-3**.

Properties of Trees

The result of the preceding theorems can be summarized to say that any arbitrary graph G is a tree if it has the following properties:

- G is connected and is circuitless, or
- G is connected and has $n - 1$ edges, or
- G is circuitless and has $n - 1$ edges, or
- There is exactly one path between every pair of vertices in G , or
- G is a minimally connected graph.

THEOREM 3-7

“In any tree (with two or more vertices), there are at least two pendant vertices”

In a tree of n vertices we have $n - 1$ edges, and hence $2(n - 1)$ degrees to be divided among n vertices.

Since no vertex can be of zero degree, we must have at least two vertices of degree one in a tree.

DISTANCE AND CENTERS IN A TREE

In a connected graph G , the distance $d(v_i, v_j)$ between two of its vertices v_i and v_j is the length of the shortest path (i.e., the number of edges in the shortest path) between them.

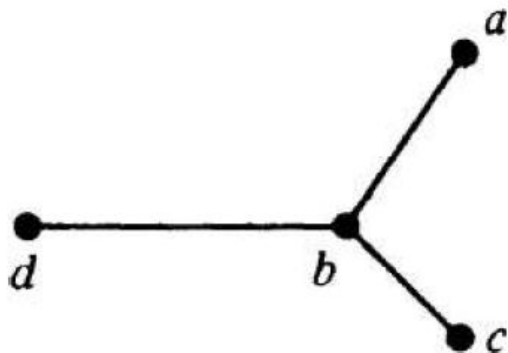
Note that this definition is valid for vertices of any graph, not just trees.

Only the difference is that in a graph that is not a tree, there are generally several paths between a pair of vertices. We have to enumerate all these paths and find the length of the shortest one.

DISTANCE AND CENTERS IN A TREE

But in a tree, there is exactly one path between any pair of vertices, so finding the distance is much easier, as it is simply the number of edges between the two vertices.

For instance, in the following tree, $d(a,b) = 1$, $d(a,c) = 2$, and $d(b,c) = 1$.



DISTANCE AS A METRIC

A function that satisfies the following three conditions is a metric

1. Nonnegativity: $f(x, y) \geq 0$, and $f(x, y) = 0$ if and only if $x = y$.
2. Symmetry: $f(x, y) = f(y, x)$.
3. Triangle inequality: $f(x, y) \leq f(x, z) + f(z, y)$ for any z .

The distance we just defined is a metric, as it trivially satisfies conditions 1 and 2. We can prove that it also satisfies condition 3.

Since $d(v_i, v_j)$ is the length of the shortest path between vertices v_i and v_j , this path cannot be longer than another path between v_i and v_j , which goes through a specified vertex v_k . Hence $d(v_i, v_j) \leq d(v_i, v_k) + d(v_k, v_j)$.

ECCENTRICITY

The eccentricity $E(v)$ of a vertex v in a graph G is the distance from v to the vertex farthest from v in G

$$E(v) = \max_{v_i \in G} d(v, v_i)$$

A vertex with minimum eccentricity in graph G is called a center of G .

There can be multiple vertices with the same minimum eccentricity, implying that there can be multiple centers, not just one.

THEOREM 3-9

“Every tree has either one or two centers”

To prove this, we start with an observation. The maximum distance, $\max d(v, v_i)$, from a given vertex v to any other vertex v_i occurs only when v_i is a pendant vertex.

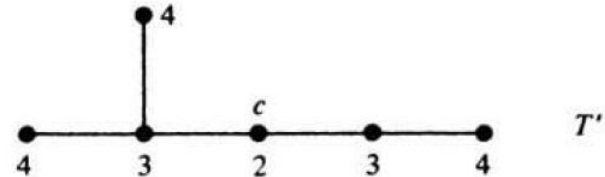
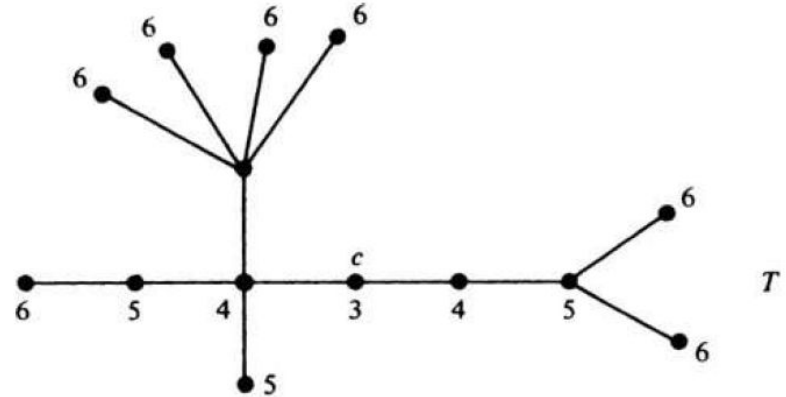
With this observation, let us start with a tree T having more than two vertices. Tree T must have two or more pendant vertices (Theorem 3-7).

THEOREM 3-9

Delete all the pendant vertices from T .
The resulting graph T' is still a tree.

What about the eccentricities of the vertices in T' ?

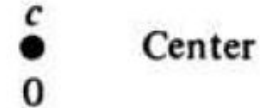
A little deliberation will reveal that removal of all pendant vertices from T uniformly reduced the eccentricities of the remaining vertices by one.



THEOREM 3-9

Therefore, all vertices that T had as centers will still remain centers in T' . From T' we can again remove all pendant vertices and get another tree T'' .

We continue this process until there is left either a vertex (which is the center of T) or an edge (whose end vertices are the two centers of T).

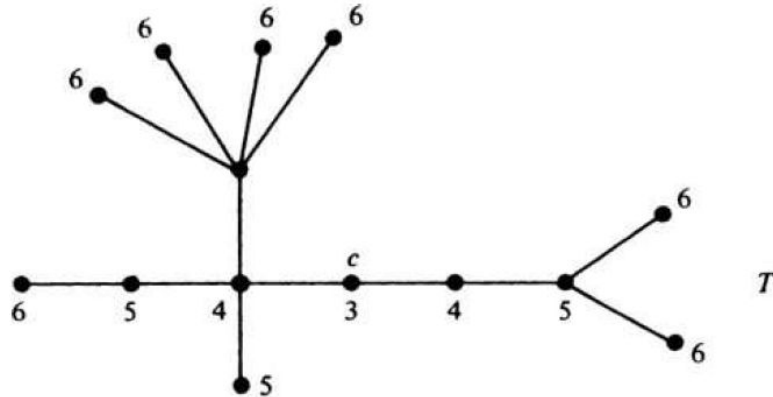


RADIUS AND DIAMETER

If a tree has a center (or two centers), does it have a radius also?

Yes.

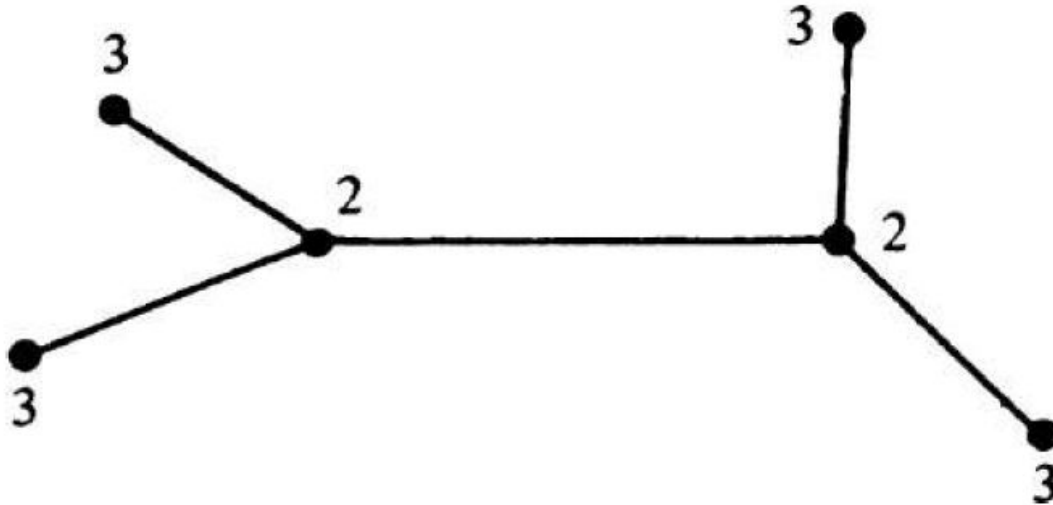
The eccentricity of a center (which is the distance from the center of the tree to the farthest vertex) in a tree is defined as the radius of the tree. For instance, the following graph has the radius 3.



RADIUS AND DIAMETER

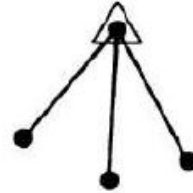
The diameter of a tree T , on the other hand, is defined as the length of the longest path in T .

Unlike circles in geometry, a radius in a tree is not necessarily half its diameter.



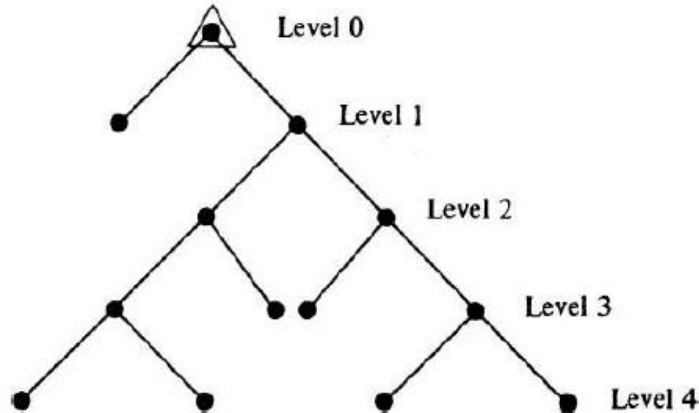
ROOTED AND BINARY TREES

A tree in which one vertex (called the root) is distinguished from all the others is called a rooted tree.



ROOTED AND BINARY TREES

A binary tree is a special class of rooted tree in which there is exactly one vertex of degree two, and each of the remaining vertices is of degree one or three (applicable for 3 or more vertices)



Since the vertex of degree two is distinct from all other vertices, this vertex serves as a root.

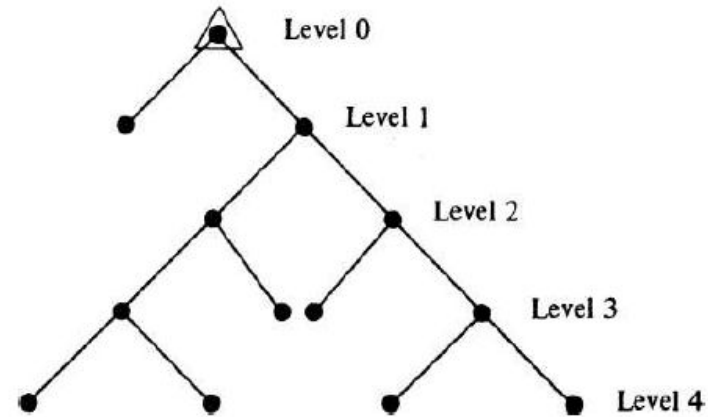
ROOTED AND BINARY TREES

Two properties of binary trees follow directly from the definition:

First, the number of vertices n in a binary tree is always odd.

This is because there is exactly one vertex of even degree, and the remaining $n - 1$ vertices are of odd degrees.

Since from **Theorem 1-1** the number of vertices of odd degrees is even, $n - 1$ is even. Hence n is odd.

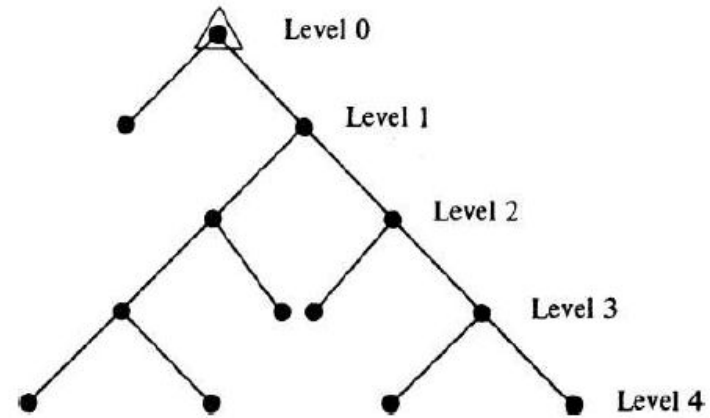


ROOTED AND BINARY TREES

Second, let p be the number of pendant vertices in a binary tree T .

Then $n - p - 1$ is the number of vertices of degree three. Therefore, the number of edges in T equals

$$\frac{p + 3(n - p - 1) + 2}{2} = n - 1$$
$$p = \frac{n + 1}{2}$$

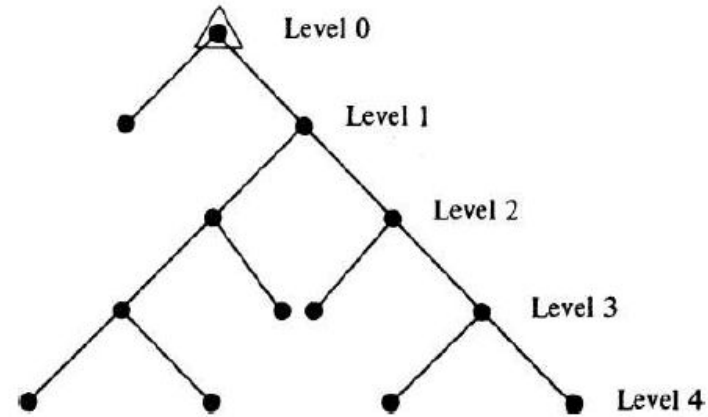


ROOTED AND BINARY TREES

A nonpendant vertex in a tree is called an internal vertex.

From our second observation, we can infer that the number of internal vertices in a binary tree is one less than the number of pendant vertices.

In a binary tree a vertex v_i is said to be at level l_i if v_i is at a distance of l_i from the root. Thus the root is at level 0.

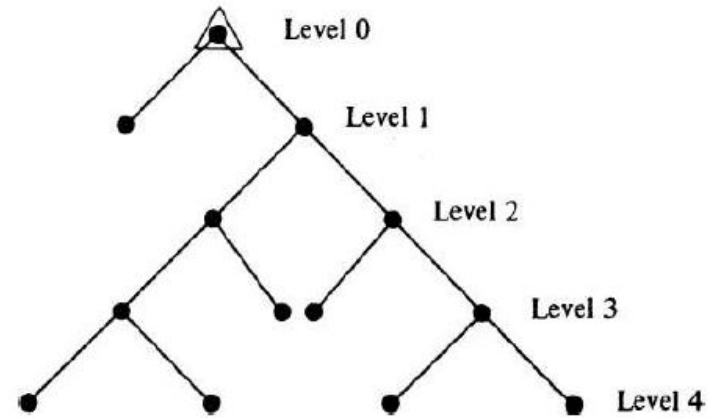


ROOTED AND BINARY TREES

Clearly, there can be only one vertex (the root) at level 0, at most two vertices at level 1, at most four vertices at level 2, and so on.

Therefore, the maximum number of vertices possible in a k-level binary tree is

$$2^0 + 2^1 + 2^2 + \dots + 2^k \geq n$$

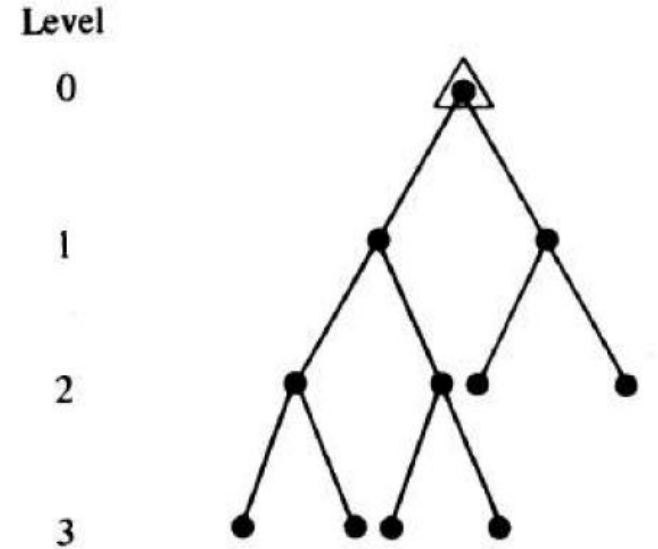


ROOTED AND BINARY TREES

The maximum level, $l_{\max}$, of any vertex in a binary tree is called the height of the tree.

The minimum possible height occurs when every level is fully filled. So, for an n -vertex tree, the minimum height is

$$\min l_{\max} = \lceil \log_2 (n + 1) - 1 \rceil$$

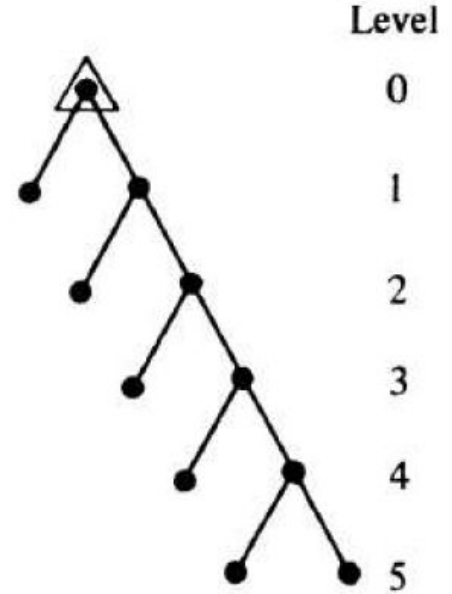


$$\min l_{\max} = \lceil (\log_2 12) - 1 \rceil$$

ROOTED AND BINARY TREES

And the maximum possible height is obtained when the farthest vertex is as far as possible from the root, so we must have exactly two vertices at each level, except at the 0 level. Therefore,

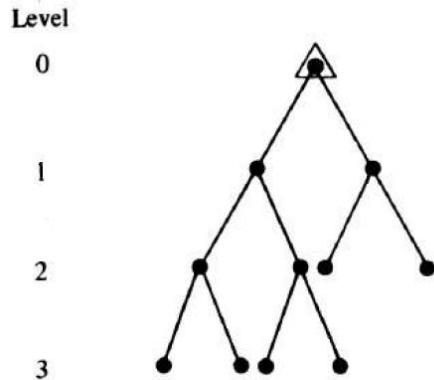
$$\max l_{\max} = \frac{n-1}{2}$$



$$\max l_{\max} = \frac{11 - 1}{2} = 5$$

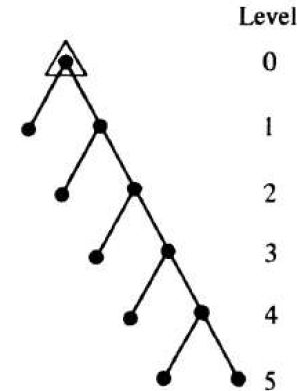
PATH LENGTH

In analysis of algorithms we are generally interested in computing the sum of the levels of all pendant vertices. This quantity, known as the **path length**, is defined as the sum of the path lengths from the root to all pendant vertices.



$$\min l_{\max} = \lceil (\log_2 12) - 1 \rceil$$

$$\text{Path length} = 3+3+3+3+2+2=16$$



$$\max l_{\max} = \frac{11-1}{2} = 5$$

$$\text{Path length} = 1+2+3+4+5+5=20$$

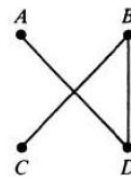
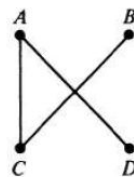
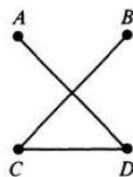
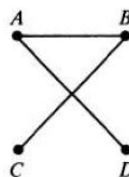
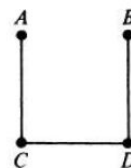
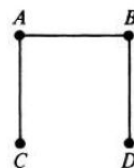
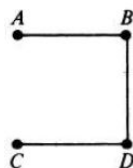
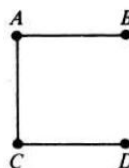
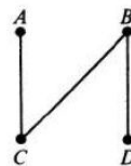
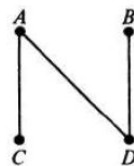
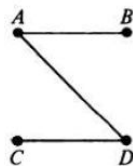
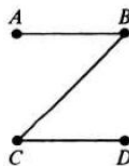
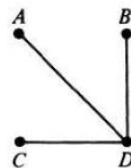
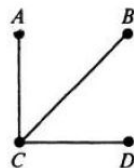
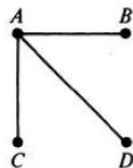
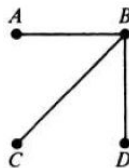
COUNTING TREES

While studying the structural isomers of saturated hydrocarbons (C_nH_{2n+2}), Arthur Cayley came up with the question:

What is the number of different trees that one can construct with n distinct (or labeled) vertices?

If $n = 4$, for instance, we have 16 trees, as shown in the following slide.

COUNTING TREES



THEOREM 3-10 (Cayley's Theorem)

“The number of labeled trees with n vertices ($n \geq 2$) is n^{n-2} .”

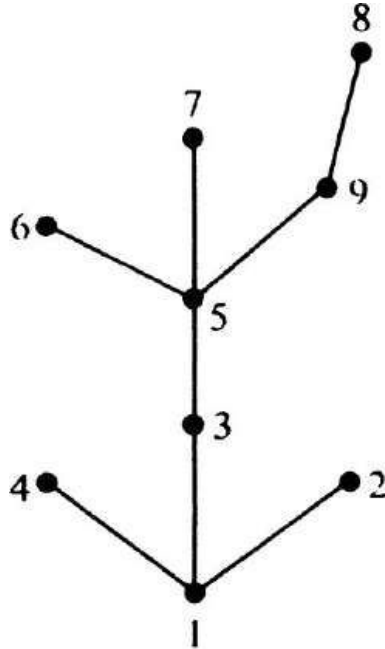
Proof: Let the n vertices of a tree T be labeled $1, 2, 3, \dots, n$.

Remove the pendant vertex (and the edge incident on it) having the smallest label, which is, say, a_1 . Suppose that b_1 was the vertex adjacent to a_1 . Among the remaining $n - 1$ vertices let a_2 be the pendant vertex with the smallest label, and b_2 be the vertex adjacent to a_2 . Remove the edge (a_2, b_2) .

This operation is repeated on the remaining $n - 2$ vertices, and then on $n - 3$ vertices, and so on. The process is terminated after $n - 2$ steps, when only two vertices are left. The tree T defines the unique sequence $(b_1, b_2, \dots, b_{n-2})$.

THEOREM 3-10 (Cayley's Theorem)

For example, applying the previously mentioned steps on this tree would result in the sequence (1,1,3,5,5,5,9)



THEOREM 3-10 (Cayley's Theorem)

Conversely, given a sequence of $n-2$ labels, an n -vertex labeled tree can be constructed as follows:

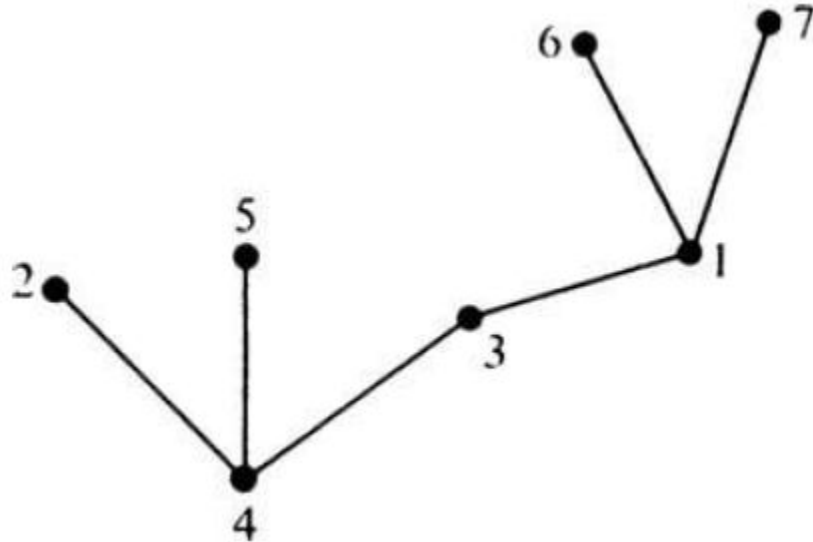
Determine the first number in the **label set** $(1, 2, \dots, n)$ that does not appear in the given **sequence**. This number clearly is a_1 . And thus the edge (a_1, b_1) is defined.

Remove b_1 from the **sequence** and a_1 from the **label set**. In the remaining **label set**, find the next smallest number that does not appear in the remainder of the **sequence**. This would be a_2 , and thus the edge (a_2, b_2) is defined.

The construction is continued till the **sequence** is empty, and the **label set** has 2 elements left, which will be finally joined by an edge.

THEOREM 3-10 (Cayley's Theorem)

For example, given a sequence $(4, 4, 3, 1, 1)$, we can construct a seven-vertex tree as follows: $(2, 4)$ is the first edge. The second is $(5, 4)$. Next, $(4, 3)$. Then $(3, 1)$, $(6, 1)$, and finally $(7, 1)$, as shown below



THEOREM 3-10 (Cayley's Theorem)

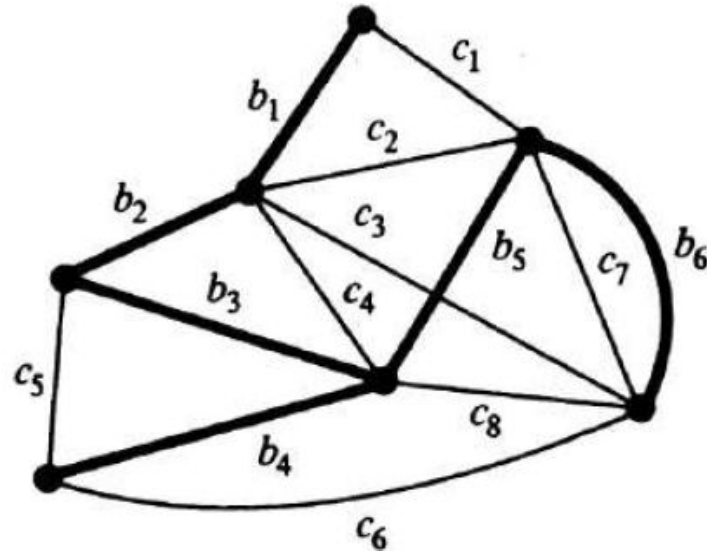
For each of the $n - 2$ elements in the given sequence, we can choose any one of n numbers, thus forming n^{n-2} tuples, each defining a distinct labeled tree of n vertices.

And since each tree defines one of these sequences uniquely, there is a one-to-one correspondence between the trees and the n^{n-2} sequences. Hence the theorem.

SPANNING TREES

A tree T is said to be a spanning tree of a connected graph G if T is a subgraph of G and T contains all vertices of G .

For instance, the subgraph in heavy lines in the following graph is a spanning tree.



SPANNING TREES

Finding a spanning tree of a connected graph G is simple.

If G has no circuit, it is its own spanning tree. If G has a circuit, delete an edge from the circuit. This will still leave the graph connected.

If there are more circuits, repeat the operation till an edge from the last circuit is deleted, leaving a connected, circuit-free graph that contains all the vertices of G .

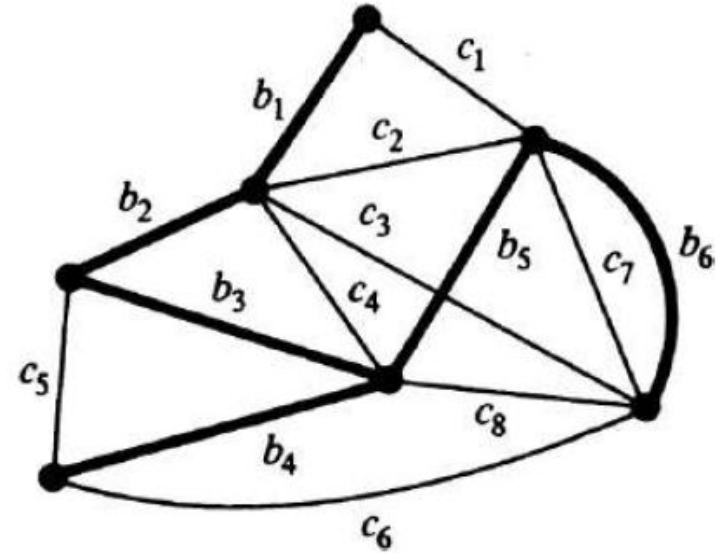
BRANCHES AND CHORDS

An edge in a spanning tree T is called a branch of T .

An edge of G that is not in a given spanning tree T is called a chord.

For instance, edges b_1, b_2, b_3, b_4, b_5 , and b_6 are branches of the spanning tree shown here, while edges $c_1, c_2, c_3, c_4, c_5, c_6, c_7$, and c_8 are chords.

It must be kept in mind that branches and chords are defined only with respect to a given spanning tree.



THEOREM 3-11 & 3-12

Based on our definitions of spanning trees, branches and chords, the following two theorems are immediately evident:

THEOREM 3-11: *Every connected graph has at least one spanning tree.*

THEOREM 3-12: *With respect to any of its spanning trees, a connected graph of n vertices and e edges has $n - 1$ tree branches and $e - n + 1$ chords.*

RANK AND NULLITY

How are the vertex number (n), edge number (e) and component number (k) of a graph related?

Since every component of a graph must have at least one vertex, $n \geq k$.

Moreover, the number of edges in a component can be no less than the number of vertices in that component minus one. Therefore, $e \geq n - k$.

Apart from the constraints $n - k \geq 0$ and $e - n + k \geq 0$, these three numbers n , e , and k are independent, and they are fundamental numbers in graphs.

RANK AND NULLITY

From these three numbers are derived two other important numbers called **rank (r)** and **nullity (μ)**, defined as follows:

$$r = n - k$$

$$\mu = e - n + k$$

The rank of a connected graph is $n - 1$, and the nullity is $e - n + 1$.

Also it may be observed here that

- rank of G = number of branches in any spanning tree (or forest) of G ,
- nullity of G = number of chords in G ,
- rank + nullity = number of edges in G .

FUNDAMENTAL CIRCUITS

If we add an edge between any two vertices of a tree, a circuit is created.

Similarly, if we consider a spanning tree T of a graph G , adding any one chord to T will create exactly one circuit. Such a circuit, formed by adding a chord to a spanning tree, is called a **fundamental circuit**.

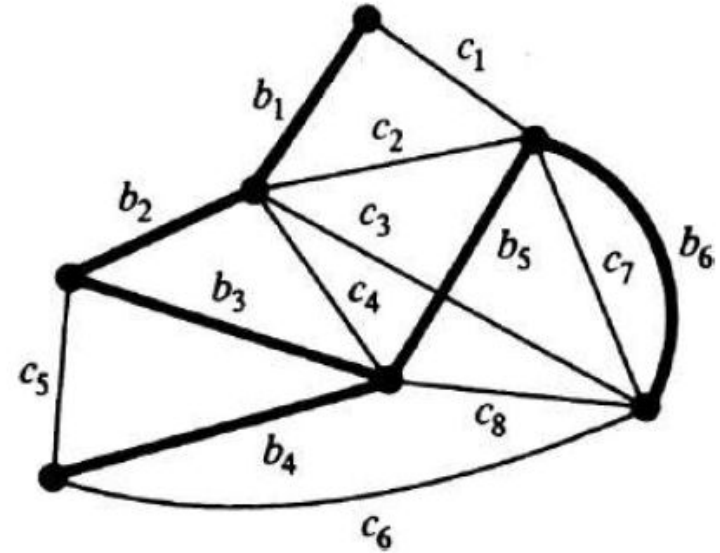
How many fundamental circuits does a graph have? Exactly as many as the number of chords (which is equal to the nullity of G)

FUNDAMENTAL CIRCUITS

Adding c_1 to it, we get a subgraph $\{b_1, b_2, b_3, b_4, b_5, b_6, c_1\}$, which has one circuit (fundamental circuit), $\{b_1, b_2, b_3, b_4, b_5, c_1\}$.

Had we added the chord c_2 (instead of c_1) to the tree, we would have obtained a different fundamental circuit, $\{b_2, b_3, b_5, c_2\}$.

Now suppose that we add both chords c_1 and c_2 to the tree. The subgraph $\{b_1, b_2, b_3, b_4, b_5, b_6, c_1, c_2\}$, has not only the fundamental circuits we just mentioned, but it has also a third circuit, $\{b_1, c_1, c_2\}$, which is not a fundamental circuit.



FUNDAMENTAL CIRCUITS

Two comments may be appropriate here.

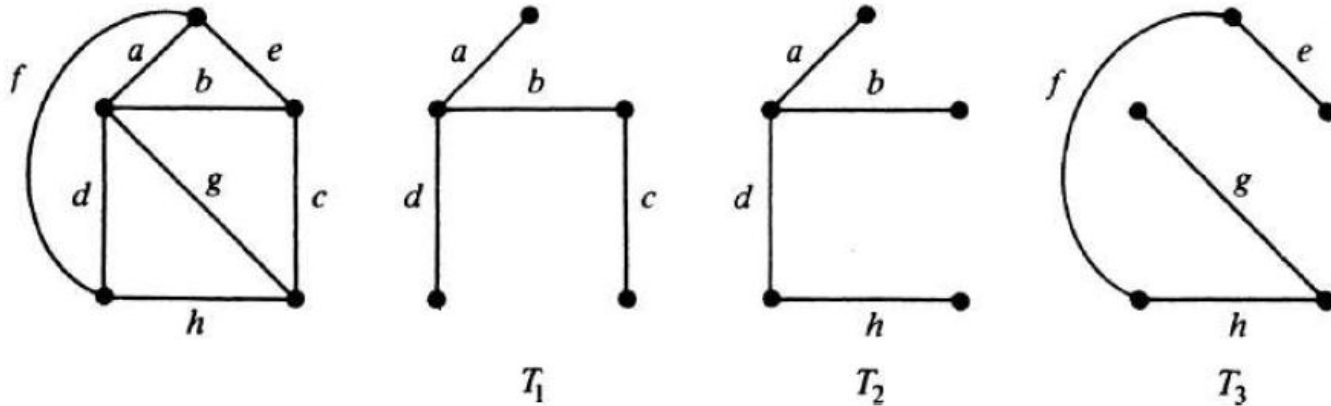
- First, a circuit is a fundamental circuit only with respect to a given spanning tree.
 - A given circuit may be fundamental with respect to one spanning tree, but not with respect to a different spanning tree of the same graph.
- Second, no matter how many circuits a network contains we need consider only fundamental circuits with respect to any spanning tree.
 - The rest of the circuits are combinations of some fundamental circuits.

FINDING ALL SPANNING TREES OF A GRAPH

For a given connected graph, there may be a large number of spanning trees.

One reasonable way to generate spanning trees of a graph is to start with a given spanning tree, say tree T_1 (a b c d in the figure).

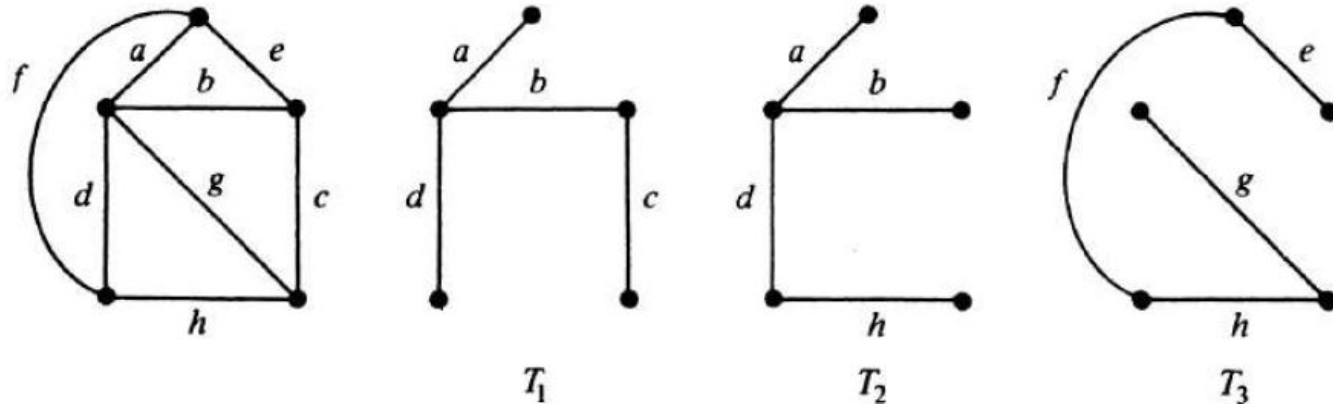
Add a chord, say h, to the tree T_1 . This forms a fundamental circuit (b c h d).



FINDING ALL SPANNING TREES OF A GRAPH

Removal of any branch, say c , from the fundamental circuit $(b\ c\ h\ d)$ just formed will create a new spanning tree T_2 .

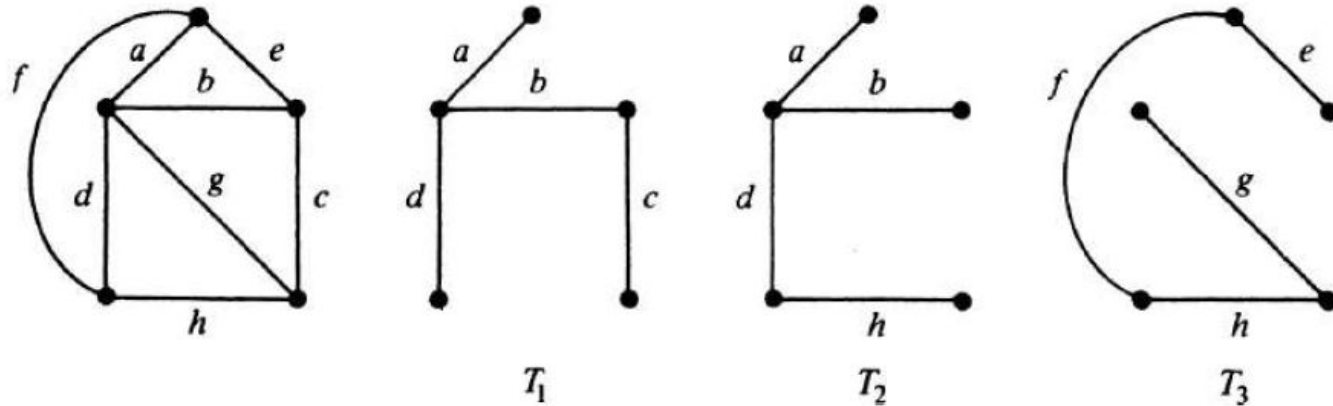
This generation of one spanning tree from another, through addition of a chord and deletion of an appropriate branch, is called a **cyclic interchange** or **elementary tree transformation**.



FINDING ALL SPANNING TREES OF A GRAPH

The distance between two spanning trees T_i and T_j of a graph G is defined as the number of edges of G present in one tree but not in the other.

This distance may be written as $d(T_i, T_j)$. For instance, in this figure, $d(T_2, T_3) = 3$

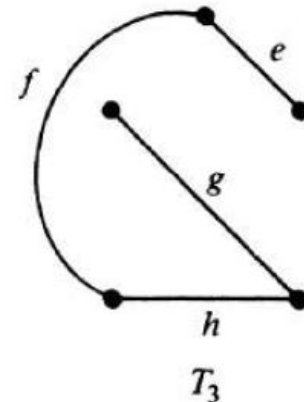
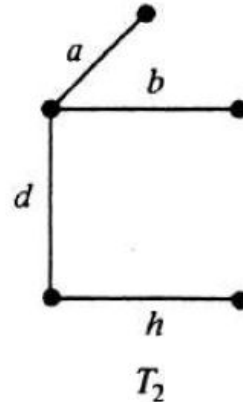
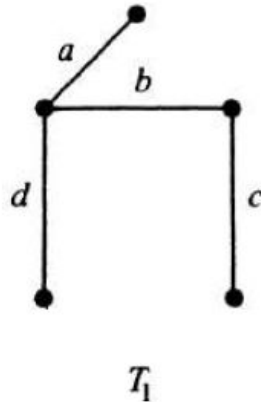
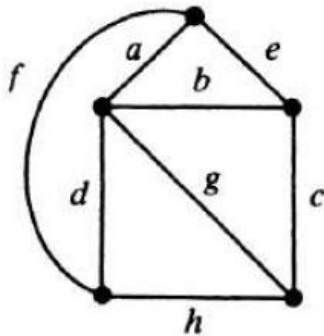


FINDING ALL SPANNING TREES OF A GRAPH

Let $T_i \oplus T_j$ be the ring sum of two spanning trees T_i and T_j of G .

$T_i \oplus T_j$ is the subgraph of G containing all edges of G that are either in T_i or in T_j but not in both). Let $N(g)$ denote the number of edges in a graph g . Then,

$$d(T_i, T_j) = \frac{1}{2} N(T_i \oplus T_j)$$



FINDING ALL SPANNING TREES OF A GRAPH

The maximum distance between any two spanning trees in G is

$$\begin{aligned}\max (d (T_i, T_j)) &= \frac{1}{2} \max (N (T_i \oplus T_j)) \\ &\leq r\end{aligned}$$

Also, if μ is the nullity of G , we know that no more than μ edges of a spanning tree T_i can be replaced to get another tree T_j , so

$$\max (d (T_i, T_j)) \leq \mu$$

Therefore, we can combinedly get $\max (d (T_i, T_j)) \leq \min (r, \mu)$

TREE GRAPHS

The tree graph of a given graph G is defined as a graph in which each vertex corresponds to a spanning tree of G , and each edge corresponds to a cyclic interchange between the spanning trees of G represented by the two end vertices of the edge.

We know that starting from any spanning tree we can obtain all other spanning trees through cyclic interchanges (or elementary tree transformations). Therefore, the tree graph of any given (finite, connected) graph is connected.

SPANNING TREES IN A WEIGHTED GRAPH

If graph G is a weighted graph, then the weight of a spanning tree T of G is defined as the sum of the weights of all the branches in T .

Among all the spanning trees of G , one with the smallest weight is called the **minimal spanning tree**.

There can be multiple minimal spanning trees in a graph, in which case one can be obtained from another by one cyclic interchange.

Algorithm for Shortest Spanning Tree

Two algorithms:

- Kruskal
- Prim

In Kruskal's algorithm, we start by listing all edges of the graph G in order of non-decreasing weight.

Next, select a smallest edge of G .

Then for each successive step select (from all remaining edges of G) another smallest edge that makes no circuit with the previously selected edges.

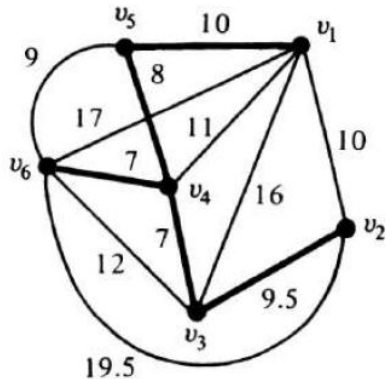
Continue until $n - 1$ edges have been selected, and these edges will constitute the desired shortest spanning tree.

Algorithm for Shortest Spanning Tree

For Prim's algorithm, draw n isolated vertices and label them $v_1, v_2, \dots, v_n$. Tabulate the given weights of the edges of G in an n by n table.

Set the weights of nonexistent edges as very large (infinity).

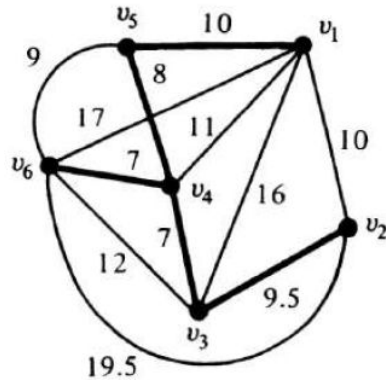
Start from vertex v_1 and connect it to its nearest neighbor (i.e., to the vertex which has the smallest entry in row 1 of the table), say v_k .



	v_1	v_2	v_3	v_4	v_5	v_6
v_1	—	10	16	11	10	17
v_2	10	—	9.5	∞	∞	19.5
v_3	16	9.5	—	7	∞	12
v_4	11	∞	7	—	8	7
v_5	10	∞	∞	8	—	9
v_6	17	19.5	12	7	9	—

Algorithm for Shortest Spanning Tree

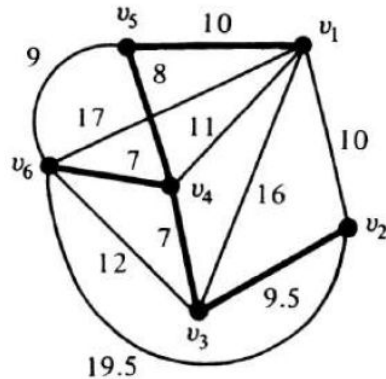
Now consider v_1 and v_k as one subgraph, and connect this subgraph to its closest neighbor (i.e., to a vertex other than v_1 and v_k that has the smallest entry among all entries in rows 1 and k). Let this new vertex be v_i .



	v_1	v_2	v_3	v_4	v_5	v_6
v_1	—	10	16	11	10	17
v_2	10	—	9.5	∞	∞	19.5
v_3	16	9.5	—	7	∞	12
v_4	11	∞	7	—	8	7
v_5	10	∞	∞	8	—	9
v_6	17	19.5	12	7	9	—

Algorithm for Shortest Spanning Tree

Next regard the tree with vertices v_1 , v_k , and v_i as one subgraph, and continue the process until all n vertices have been connected by $n - 1$ edges.



	v_1	v_2	v_3	v_4	v_5	v_6
v_1	—	10	16	11	10	17
v_2	10	—	9.5	∞	∞	19.5
v_3	16	9.5	—	7	∞	12
v_4	11	∞	7	—	8	7
v_5	10	∞	∞	8	—	9
v_6	17	19.5	12	7	9	—